ХМЕЛЬНИЦЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ

ЗВІТ З ЛАБОРАТОРНОЇ РОБОТИ № 2

Дисципліна «Кросплатформне програмування»

Розроблення об’єктної моделі застосунку

Варіант 8

Виконав: студент (впишіть ПІБ, групу)

Перевірив: Денисюк Дмитро Олександрович

Хмельницький — 2026

Мета роботи

Налаштувати Java-проєкт, реалізувати класи з інкапсульованими даними, продемонструвати запуск і перевірки. Структура відповідає методичним матеріалам кафедри.

Програмне забезпечення

Передбачено: Eclipse Temurin JDK 25, Apache NetBeans 31 (рекомендовано), Maven Wrapper 3.9.16, JUnit Jupiter, Git 2.x. Локальна перевірка в середовищі підготовки: OpenJDK 21, javac і консольний запуск. Maven/JUnit тут не запускався, тому BUILD SUCCESS не заявляється.

Постановка задачі

Створити предметну модель TaskFlow: User, Project, Task та політику завершення CompletionPolicy. Варіант 8: каталог музейних експонатів. u — кількість описів, d — кількість описів до уточнення, a — логічний параметр, який не впливає на правило.

Правило варіанта 8

allows(u,d,a) = (u ≥ 12) && (d ≤ u / 4). Цілочисельне ділення та відмова від множення d×4 уникають переповнення. Приклади: (12,3,false) — true; (12,4,true) — false; (11,0,true) — false.

Зв’язки та інваріанти

User володіє Project; Project є батьківською сутністю Task; Task має призначеного User і CompletionPolicy. Усі ідентифікатори > 0; назви не порожні; дата due ≥ created; NEW спочатку без виконавця. До статусу ACTIVE можна перейти тільки після assign. DONE — термінальний статус.

Таблиця операцій

Операція	Передумови	Результат
assign(user)	NEW, user не null	NEW; новий виконавець
start()	NEW, виконавця призначено	ACTIVE
complete(date,u,d,a)	ACTIVE, коректні аргументи, policy=true	DONE

Структура проєкту

Редаговані User.java, Project.java, Task.java, VariantPolicy.java, App.java, variant.txt, README.md. Викладацькі DomainTest.java, CounterTest.java, cases.csv, Status.java, CompletionPolicy.java, Counter.java, pom.xml та Maven Wrapper залишаються незмінними й підтягуються з GitHub.

Лістинг User.java

```
package ua.khnu.kpp;
public final class User {
    private final long id;
    private final String name;
    public User(long id, String name) {
        if (id &lt;= 0) throw new IllegalArgumentException("user id must be positive");
        if (name == null || name.isBlank()) throw new IllegalArgumentException("user name is
required");
        this.id = id;
        this.name = name;
    }
    public long id() { return id; }
    public String name() { return name; }
    @Override public boolean equals(Object other) {
        return this == other || other instanceof User user &amp;&amp; this.id == user.id;
    }
    @Override public int hashCode() { return Long.hashCode(id); }
}
```

Лістинг Project.java

```
package ua.khnu.kpp;
public final class Project {
    private final long id;
    private final String title;
    private final User owner;
    public Project(long id, String title, User owner) {
        if (id &lt;= 0) throw new IllegalArgumentException("project id must be positive");
        if (title == null || title.isBlank()) throw new IllegalArgumentException("project title is
required");
        if (owner == null) throw new IllegalArgumentException("project owner is required");
        this.id = id;
        this.title = title;
        this.owner = owner;
    }
    public long id() { return id; }
    public String title() { return title; }
    public User owner() { return owner; }
}
```

Лістинг Task.java

```
package ua.khnu.kpp;
import java.time.LocalDate;
public final class Task {
    private final long id;
    private final String title;
    private final Project project;
    private final LocalDate created;
    private final LocalDate due;
    private final CompletionPolicy policy;
    private Status status;
    private User assignee;

    public Task(long id, String title, Project project, LocalDate created,
                LocalDate due, CompletionPolicy policy) {
        if (id &lt;= 0) throw new IllegalArgumentException("task id must be positive");
        if (title == null || title.isBlank()) throw new IllegalArgumentException("task title is
required");
        if (project == null || created == null || due == null || policy == null)
            throw new IllegalArgumentException("project, dates and policy are required");
        if (due.isBefore(created)) throw new IllegalArgumentException("due date is before
creation");
        this.id = id;
        this.title = title;
        this.project = project;
        this.created = created;
        this.due = due;
        this.policy = policy;
        this.status = Status.NEW;
        this.assignee = null;
    }
    public long id() { return id; }
    public String title() { return title; }
    public Project project() { return project; }
    public LocalDate created() { return created; }
    public LocalDate due() { return due; }
    public Status status() { return status; }
    public User assignee() { return assignee; }
```

```
    public void assign(User user) {
        if (status != Status.NEW) throw new IllegalStateException("assignment only allowed in NEW");
        if (user == null) throw new IllegalArgumentException("assignee is required");
        assignee = user;
    }
    public void start() {
        if (status != Status.NEW) throw new IllegalStateException("start only allowed in NEW");
        if (assignee == null) throw new IllegalStateException("assign a user first");
        status = Status.ACTIVE;
    }
    public void complete(LocalDate date, int units, int defects, boolean approved) {
        if (status != Status.ACTIVE) throw new IllegalStateException("completion only allowed in
ACTIVE");
        if (date == null || date.isBefore(created))
            throw new IllegalArgumentException("completion date is invalid");
        if (units &lt; 0 || defects &lt; 0)
            throw new IllegalArgumentException("evidence values must be non-negative");
        if (!policy.allows(units, defects, approved))
            throw new IllegalStateException("completion policy rejected evidence");
        status = Status.DONE;
    }
}
```

Лістинг VariantPolicy.java

```
package ua.khnu.kpp;
/** Variant 8: museum catalog; approved is informational. */
public final class VariantPolicy implements CompletionPolicy {
    @Override public boolean allows(int units, int defects, boolean approved) {
        return units >= 12 &amp;&amp; defects &lt;= units / 4;
    }
}
```

Лістинг App.java

```
package ua.khnu.kpp;
import java.time.LocalDate;
public final class App {
    private App() { }
    public static void main(String[] args) {
        User curator = new User(1, "Олена — куратор музею");
        User researcher = new User(2, "Ігор — каталогізатор");
        Project museum = new Project(1, "Каталог музейної колекції", curator);
        LocalDate created = LocalDate.of(2026, 9, 10);
        Task task = new Task(1, "Опис експонатів", museum,
                             created, created.plusDays(7), new VariantPolicy());
        System.out.println("Початковий стан: " + task.status());
        task.assign(researcher);
        task.start();
        System.out.println("Після start: " + task.status());
        try {
            task.complete(created.plusDays(5), 12, 4, false);
        } catch (IllegalStateException exception) {
            System.out.println("Відмова правила: " + exception.getMessage());
            System.out.println("Стан після відмови: " + task.status());
        }
        task.complete(created.plusDays(5), 12, 3, false);
        System.out.println("Після успішного завершення: " + task.status());
    }
}
```

Фактичний консольний запуск

```
Початковий стан: NEW
Після start: ACTIVE
Відмова правила: completion policy rejected evidence
Стан після відмови: ACTIVE
Після успішного завершення: DONE
```

План тестування

Сценарій	Очікувано
u=12,d=3,a=false	policy=true, DONE

Сценарій	Очікувано
u=12,d=4,a=true	IllegalStateException, ACTIVE
u=11,d=0,a=true	IllegalStateException, ACTIVE
complete з NEW	IllegalStateException, NEW
Дати до created	IllegalArgumentException, ACTIVE
assign у DONE	IllegalStateException, DONE

Команди перевірки

```
.\mvnw.cmd clean verify
java -cp target/classes ua.khnu.kpp.App
git diff --no-index --exit-code ../lab-2-teacher/src/test src/test
```

Здача через Git

Після запуску 14 оригінальних тестів викладача необхідно опублікувати окремий репозиторій/гілку lab-2, зафіксувати URL та справжній SHA і підтвердити незмінність тестів. URL: [заповнити]; SHA: [заповнити].

Висновок

Реалізовано модель станів з атомарними перевірками, стабільною ідентичністю користувача, незмінними атрибутами сутностей і політикою музейного каталогу. Компіляція та демонстраційний запуск перевірені локально; остаточний статус Maven/JUnit має бути отриманий після додавання незмінених викладацьких файлів.